



RV College of
Engineering®

UNIT 4

Energy Based Models

By: Prof. Rajesh R M

Assistant Professor
Dept. of AI and ML
RVCE, Bangalore

Go, change the world®



Energy-Based Models are a broad class of generative model that borrow a key idea from modeling physical systems—namely, that the **probability of an event can be expressed using a Boltzmann distribution**, a specific function that normalizes a realvalued energy function between 0 and 1.

This distribution was originally formulated in 1868 by Ludwig Boltzmann, who used it to describe gases in thermal equilibrium.

Energy-based models attempt to model the true data-generating distribution using a *Boltzmann distribution*.

where $E(x)$ is known as the *energy function* (or *score*) of an observation x .

$$p(\mathbf{x}) = \frac{e^{-E(\mathbf{x})}}{\int_{\hat{\mathbf{x}} \in \mathbf{X}} e^{-E(\hat{\mathbf{x}})}}$$



In practice, this amounts to training a neural network E to output low scores for likely observations (so p is close to 1) and high scores for unlikely observations (so p is close to 0).

$$p(\mathbf{x}) = \frac{e^{-E(\mathbf{x})}}{\int_{\hat{\mathbf{x}} \in \mathbf{X}} e^{-E(\hat{\mathbf{x}})} d\hat{\mathbf{x}}}$$

There are two challenges with modeling the data in this way

Firstly, it is not clear how we should use our model for sampling new observations—we can use it to generate a score given an observation, but how do we generate an observation that has a low score.

Secondly, the normalizing denominator of Equation contains an integral that is intractable for all but the simplest of problems. If we **cannot calculate this integral**, then we **cannot use maximum likelihood estimation to train the model**, as this requires that **p is a valid probability distribution**.

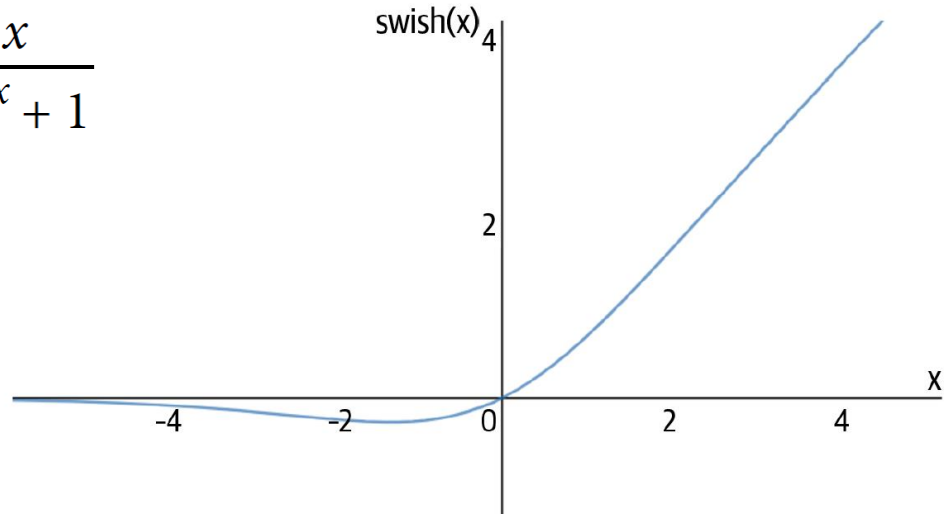


The Energy Function

The energy function **$E\vartheta x$** is a neural network with parameters ϑ that can transform an input image x into a scalar value. Throughout this network, we make use of an activation function called **swish**, as described in the following sidebar.

Swish is an alternative to ReLU that was introduced by Google in 2017 and is defined as follows:

$$\text{swish}(x) = x \cdot \text{sigmoid}(x) = \frac{x}{e^{-x} + 1}$$





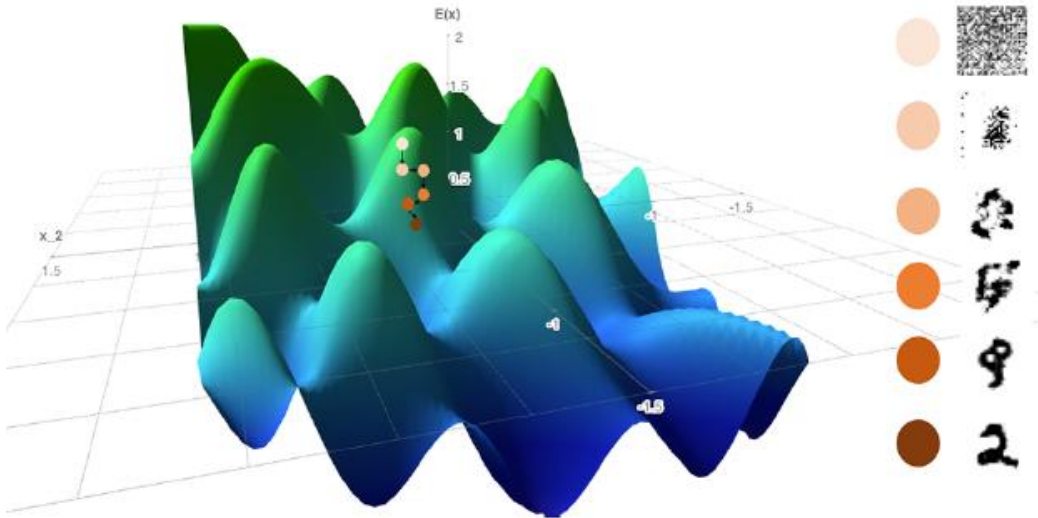
- **Sampling Using Langevin Dynamics**
- The energy function only outputs a score for a given input—how can we use this function to generate new samples that have a low energy score?
- ***Langevin dynamics*** *Techniques used*, which makes use of the fact that we can compute the gradient of the energy function with respect to its input.
- If we start from a random point in the sample space and take small steps in the opposite direction of the calculated **gradient**, we will gradually reduce the energy function. If our neural network is trained correctly,
- the random noise should transform into an image that resembles an observation from the training set before our eyes!



RV College of
Engineering®

EBM

Go, change the world®



Stochastic Gradient Langevin Dynamics

Importantly, we must also add a small amount of random noise to the input as we travel across the sample space; otherwise, there is a risk of falling into local minima. The technique is therefore known as **stochastic gradient Langevin dynamics**.



Stochastic Gradient Langevin Dynamics

When training a neural network, we calculate the **gradient of the *loss function*** with respect to the ***parameters*** of the network (i.e., the weights) using **backpropagation**.

Then we update the parameters a small amount in the **direction of the negative gradient**, so that over many iterations, we gradually **minimize the loss**.

With Langevin dynamics, we keep the neural **network weights *fixed*** and calculate the gradient of the ***output*** with respect to the ***input***.

Then we update the input a small amount in the direction of the negative gradient, so that over many iterations, we gradually minimize the output (the energy score).

Both processes utilize the same idea (gradient descent), but are applied to different functions and with respect to different entities.



Formally, Langevin dynamics can be described by the following equation:

$$x^k = x^{k-1} - \eta \nabla_x E_\theta(x^{k-1}) + \omega$$

where $\omega \sim 0, \sigma$ and $x_0 \sim (-1,1)$. η is the step size **hyperparameter** that must be tuned—too large and the steps jump over minima, too small and the algorithm will be too slow to converge.

$x_0 \sim (-1,1)$ is the uniform distribution on the range $[-1, 1]$.